

Heaven's light is our guide"

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering

Compiler Design

Course No. : CSE 701

Chapter 6: Type Checking

Prepared By : Julia Rahman

Overview

Static Checking:

- ✓ Check that the source program follows both the syntactic and semantic conventions of the source language
- ✓ Examples of static check
 - **Type checks:**
 - A compiler should report an error if an operator is applied to an incompatible operand
 - For example, if an array variable and a function variable are added together.
 - **Flow-of-control checks:**
 - Statements that cause flow of control to leave a construct must have some place to which to transfer the flow of control.
 - a break statement in C not enclosed within a while, for, or, switch statement, an error occurs.
 - **Uniqueness checks:**
 - Each variable should only be declared once.
 - uniquely declared identifier
 - **Name-related checks**
 - Same name appears two or more times.

Overview

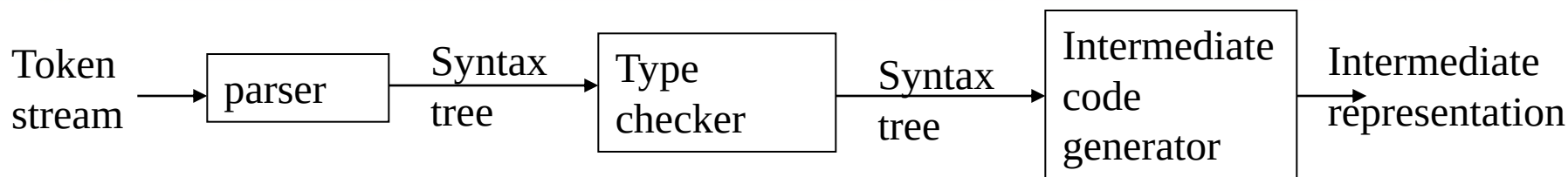


Figure 6.1 Position of type checker

- ✓ TYPE CHECKING is the main activity in semantic analysis.
- ✓ **Goal:** calculate and ensure consistency of the type of every expression in a program
- ✓ If there are type errors, we need to notify the user.
- ✓ Otherwise, we need the type information to generate code that is correct.

Notes:

- 1) A type checker verifies that the type of a construct matches that expected by its context.
- 2) Type information gathered by a type checker may be needed when code is generated.
- 3) A symbol that can represent different operations in different context is said to be “overloaded”

6.1 Type Systems

Type expressions:

Denote the type of a language construct

- 1) A basic type is a type expression. The basic types are boolean, char, integer, and real. The special basic type `type_error` signifies an error. The special type `void` signifies “no type”
- 2) A type name is a type expression. (type names are like typedefs in C)
- 3) A type constructor applied to type expression is a type expression.

Constructors include:

- a. **Arrays:** if T is a type expression, then `pointer( $T$ )` is a type expression denoting the type “pointer to an object of type T ” `Array( $I, T$ )` $\leftarrow I$: index set, T : element type
- b. **Products:** if T_1 and T_2 are type expressions, then their Cartesian product $T_1 \times T_2$ is also a type expression.
- c. **Records:** a record is a special kind of product in which the fields have names (examples below)
- d. **Pointers:** if T is a type expression, then `pointer( $T$ )` is a type expression denoting the type “pointer to an object of type T ”
- e. **Functions:** functions map elements of a domain D to a range R , so we

6.1 Type Systems

write $D \rightarrow R$ to denote “function mapping objects of type D to objects of type R ” (examples below)

- 4) Type expressions may contain variables whose values are type expressions.

Notes: A convenient way to represent a type expression is to use a graph

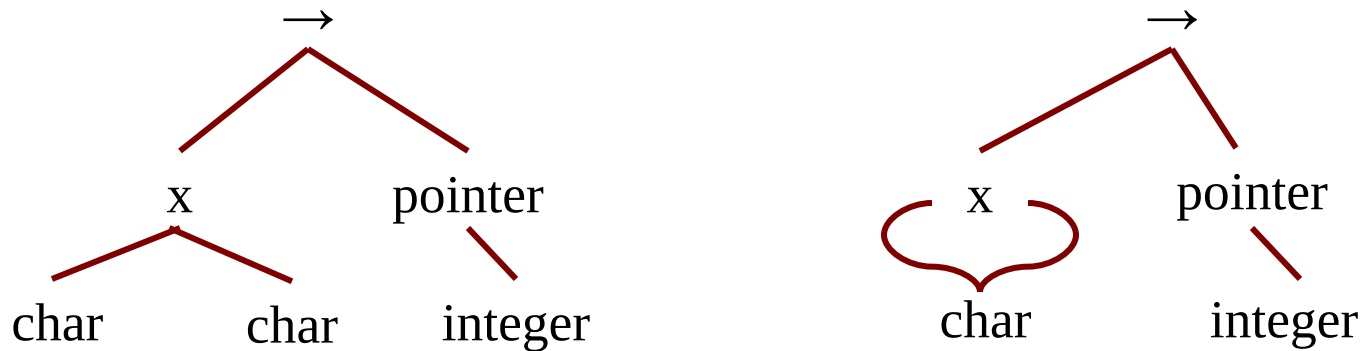


Figure 6.2 Tree and Dag respectively for $(\text{char } x \text{ char}) \rightarrow \text{pointer}(\text{integer})$

6.1 Type Systems

Type systems:

- ✓ A type system is a collection of rules for assigning type expressions to the various parts of a program.
- ✓ Every type checker implements some type system.
- ✓ The type systems are specified in a syntax-directed manner
- ✓ Different type systems may be used by different compilers or processors of the same language
- ✓ Syntax-directed type checking is a simple method to implement a type checker.

Static and Dynamic Checking of Types

- ✓ STATIC type checking is done at compile time.
- ✓ DYNAMIC type checking is done at run time.
- ✓ If the compiler guarantees that every program it accepts will run without type errors, then the language is **STRONGLY TYPED**.

6.2 Specification of a Simple Type Checker

Type checking for expressions:

Once the identifiers and their types have been inserted into the symbol table, we can check the type of the elements of an expression:

- [illegible]

6.2 Specification of a Simple Type Checker

Type checking for statements:

- ✓ Usually we assign the type VOID to statements.
- ✓ If a type error is found during type checking, though, we should set the type to `type_error`
- ✓ Let's change our grammar allow statements:

$P \rightarrow D ; S$

i.e., a program is a sequence of declarations followed by a sequence of statements.

- ✓ Now we need to add productions and semantic actions:

- | | |
|---|---|
| ▪ $S \rightarrow \text{id} := E$ | { if <code>id.type</code> = <code>E.type</code> then <code>S.type</code> := <code>void</code>
else <code>S.type</code> := <code>type_error</code> } |
| ▪ $S \rightarrow \text{if } E \text{ then } S_1$ | { if <code>E.type</code> = <code>boolean</code> then <code>S.type</code> := <code>S₁.type</code>
else <code>S.type</code> := <code>type_error</code> } |
| ▪ $S \rightarrow \text{while } E \text{ do } S_1$ | { if <code>E.type</code> = <code>boolean</code> then <code>S.type</code> := <code>S₁.type</code>
else <code>S.type</code> := <code>type_error</code> } |
| ▪ $S \rightarrow S_1 ; S_2$ | { if <code>S₁.type</code> = <code>void</code> and <code>S₂.type</code> = <code>void</code> then <code>S.type</code> := <code>void</code>
else <code>S.type</code> := <code>type_error</code> . |

6.2 Specification of a Simple Type Checker

Type checking for functions:

- ✓ Suppose we add a production $E \rightarrow E (E)$
- ✓ Then we need productions for function declarations:

$T \rightarrow T1 \rightarrow T2$	$\{ T.type := T1.type \rightarrow T2.type \}$
-----------------------------------	---

and function calls:

$E \rightarrow E1 (E2)$	$\{ \text{if } E2.type = s \text{ and } E1.type = s \rightarrow t$ $\text{then } E.type := t$ $\text{else } E.type := type_error \}$
---------------------------	---

6.3 Structural Equivalence of type expression



What does it mean for two type expressions to be equal?

- ✓ **Structural Equivalence** says two types are the same if they are made up of the same basic types and constructors.
- ✓ **Name equivalence** says two types are the same if their constituents have the SAME NAMES.



When type expressions are represented by graphs, two types are **structurally equivalent** if and only if one of the following conditions is true:

- 1) They are the same basic type.
- 2) They are formed by applying the same constructor to structurally equivalent types.
- 3) One is a type name that denotes the other .



Example: type vector = array [1..10] of real
 type weight = array [1..10] of real
 var x, y: vector; z: weight

Name Equivalence: When they have the same name. x, y have the same type; z has a different type.

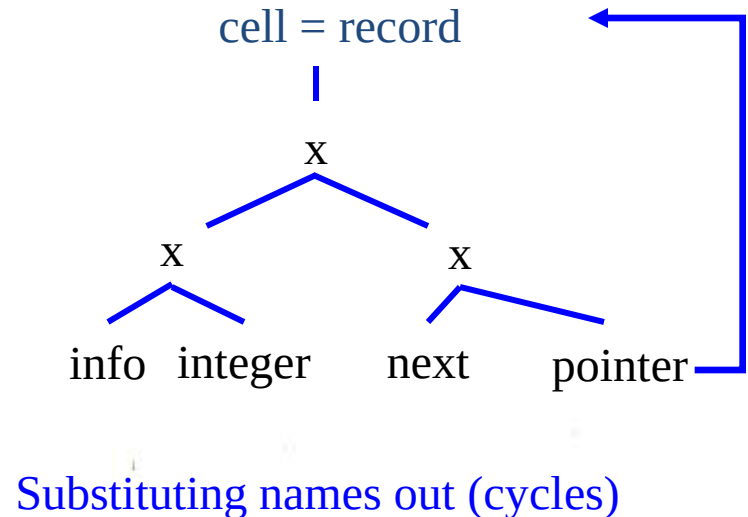
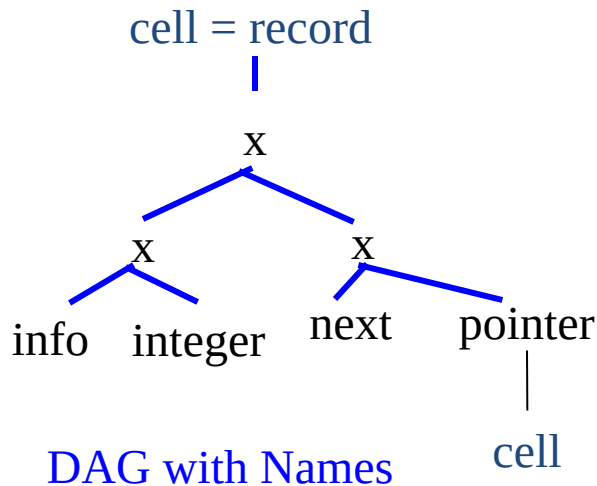
Structural Equivalence: When they have the same structure. x, y, z have the same type.

6.3 Structural Equivalence of type expression

Recursive Types

- ✚ **Where:** Linked Lists, Trees, etc.
- ✚ **How:** records containing pointers to similar records
- ✚ **Example:**
type link = $\uparrow$ cell;
cell = record
 info: integer;
 next : link
end;

✚ **Representation:**



6.4 Type conversion

- ✓ Suppose we encounter an expression $x + i$ where x has type real and i has type integer.
- ✓ CPU instructions for addition could take EITHER float OR integer as operands, but not a mix.
- ✓ This means the compiler must sometimes convert the operands of arithmetic expressions to ensure that operands are consistent with operators.
- ✓ With postfix as an intermediate language for expressions, we could express the conversion as follows:

$x \ i \ \text{intto} \ \text{real} +$

where $\text{real} +$ is the floating point addition operation.

Coercions:

- ✓ If type conversion is done by the compiler without the programmer requesting it, it is called IMPLICIT conversion or type COERCION.
- ✓ EXPLICIT conversions are those that the programmer specifies, e.g.
$$x = (\text{int})y * 2;$$
- ✓ Implicit conversion of CONSTANT expressions should be done at compile time.

6.4 Type conversion

Example 6.5:

<u>Production</u>	<u>Semantic Rule</u>
▪ $E \rightarrow \text{num}$	$E.\text{type} := \text{integer}$
▪ $E \rightarrow \text{num} . \text{num}$	$E.\text{type} := \text{real}$
▪ $E \rightarrow \text{id}$	$E.\text{type} := \text{lookup}(\text{id}.\text{entry})$
▪ $E \rightarrow E_1 \text{ op } E_2$	$E.\text{type} :=$ if $E_1.\text{type} = \text{integer}$ and $E_2.\text{type} = \text{integer}$ then integer else if $E_1.\text{type} = \text{integer}$ and $E_2.\text{type} = \text{real}$ then real else if $E_1.\text{type} = \text{real}$ and $E_2.\text{type} = \text{integer}$ then real else if $E_1.\text{type} = \text{real}$ and $E_2.\text{type} = \text{real}$ then real else type_error

Figure 6.9: Type-checking rules for coercion from integer to real